Singly Linked List –
Length of Linked List
– Space Complexity and Algorithm.

Program: –

```
int lengthOfList(Node **head)
{

    int length = 0;
    Node *current = *head;

    while (current != nullptr)
    {
        length++;
        current = current->next;
    }
    return length;
}
….
case 17:
        {
            cout << "The length of the list is: " <<lengthOfList(&head) << endl;
            break;
        }
```

Space Complexity Analysis

1. Input Space:

A. Parameter – Only Definition:

*The input space complexity refers to the space required for the input parameters of the function. In this case, the function takes a pointer to a pointer (Node ** head) as an argument. This requires a constant amount of space, which is* $O(1)$.

This view only considers the memory for the parameters passed to the function. The single parameter `Node **head` is a pointer-to-a-pointer, which has a **fixed, constant size**. Therefore, the input space complexity is **O(1)**.

• head (type Node**) is a single machine-word pointer.

⇒ Θ(1) = O(1)

$B. Full - Data\ (inclusive) Definition$:

This more common definition includes the entire data structure. If the linked list has **n nodes**, the space required to store that list is proportional to `n`. Therefore, the input space complexity is **Θ(n)**.

- **Same single word as above (Θ(1)), plus**
- **Every node reachable from *head when the function starts.**

If the list contains n nodes, they occupy Θ(n) bytes.

⇒ Θ(n) = O(n)

2. *Auxiliary Space Complexity*

Auxiliary space complexity refers to the extra space used by the algorithm that is not part of the input. In the case of the lengthOfList function, the auxiliary space is determined by the variables declared within the function:

- ***int length: This variable is used to count the number of nodes in the list.***
- ***Node * current: This pointer is used to traverse the linked list.***

The function uses a single integer variable length to keep track of the count of nodes, which requires O(1) space.

The pointer current is also declared to traverse the linked list, which again requires O(1) space.

Auxiliary space is the extra memory an algorithm uses, not counting the input. For this function:

1. `int length`: The function declares a single integer to act as a counter. An integer occupies a constant amount of memory.
2. `Node *current`: It declares a single pointer to traverse the list. This pointer also occupies a constant amount of memory.
3. **Iteration**: The `while` loop reuses these two variables. No new memory is allocated during the counting process.

Since the function only needs a fixed number of extra variables (one integer and one pointer), the amount of extra memory used is constant, regardless of the list's length.

Therefore, the auxiliary space complexity is **Θ(1)**.

- $\boldsymbol{Local\ stack\ variables}$:

  $-\ \boldsymbol{int\ length}$

  $-\ \boldsymbol{Node\ *current}$

  $-\ \boldsymbol{a\ few\ compiler-generated\ temporaries}$

  $\boldsymbol{Together\ they\ consume\ constant\ memory}.$

- $\boldsymbol{No\ heap\ allocations, no\ recursion}.$

$\Rightarrow\ \boldsymbol{\Theta}(\mathbf{1})\ =\ \boldsymbol{O}(\mathbf{1})$

$\boldsymbol{Algorithm}$

$\boldsymbol{lengthOfList(START)}$

$\mathbf{1}.\boldsymbol{SET\ Count} := \mathbf{0}$

$\mathbf{2}.\boldsymbol{SET\ CURRPTR} := \boldsymbol{START}\ [\boldsymbol{CURRPTR\ will\ point\ to\ Starting\ NODE}]$

$\mathbf{3}.\boldsymbol{Repeat\ while\ CURRPTR} \neq \boldsymbol{NULL}, \boldsymbol{then}:$

$\boldsymbol{SET\ Count} := \boldsymbol{Count} + \mathbf{1}\ [\boldsymbol{Update\ and\ increment\ COUNT}]$

$\boldsymbol{SET\ CURRPTR} := \boldsymbol{LINK}[\boldsymbol{CURRPTR}]\ \ [\boldsymbol{CURRPTR\ will\ now\ point\ to\ Next\ NODE}]$

$[\boldsymbol{End\ of\ LOOP\ structure}]$

$\mathbf{4}.\boldsymbol{RETURN\ Count}.$

$[\boldsymbol{End\ of\ Algorithm}]$